REPORTE DE PRÁCTICA: Memorias e Instrucciones



Universidad de Guadalajara

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Arquitectura de Computadoras

12/10/2026

Yael Alejandro Santero Mazano
Cabrera Balderas Jose Nery
Geronimo Sanchez Martin Alejandro

1. INTRODUCCIÓN

Diseñar, la integración modular y la comprobación funcional del sistema digital RORO Supreme. El análisis de la arquitectura de procesadores, al articular unidades aritmético-lógicas (ALU) con subsistemas de memoria ROM para la decodificación y ejecución secuencial de instrucciones de 12 bits.

Objetivos Generales:

- Diseñar una memoria ROM de instrucciones parametrizada con una capacidad de 16 palabras de 12 bits.
- Instanciar y parametrizar memorias ROM de operandos de 32 bits (ROMa y ROMina), y realizar su lectura dinámica desde archivos externos.
- Integrar el módulo ALUMIPS para ejecutar operaciones de suma, resta y comparación (SLT).
- Desarrollar un testbench automatizado en Verilog que permita validar de forma determinista el valor de salida y el estado de la bandera ZF.

2. CÓDIGOS

RORO.v

```
module RORO(
    input [3:0] posicionOp,
    output [31:0] resultado,
    output ZF
);
    wire [11:0] instruccionOp;
    wire [31:0] c1, c2;

    RORO_Suprime RORO_SuPrima(
        .posicion(posicionOp),
        .instruccion(instruccionOp)
    );

    ROma Roma(
        .DirL(instruccionOp[7:4]),
        .Data(c1)
    );

    ROmina Romina(
        .DirL(instruccionOp[3:0]),
        .Data(c2)
    );

    ALU Pablo(
        .ALU_op(instruccionOp[11:8]),
        .A(c1),
```

```
        .B(c2),
        .result(resultado),
        .zero(ZF)
    );
endmodule
```

RORO_tb.v

```
`timescale 1ns/1ns
module RORO_Tb();

reg[3:0] posicion_tb;
wire[31:0] resultado_tb;
wire ZF_tb;

RORO DUV(
      .posicionOp(posicion_tb),
      .resultado(resultado_tb),
      .ZF(ZF_tb)
);

initial
begin
      posicion_tb = 4'b0000;
      #100;

      posicion_tb = 4'b0001;
      #100;

      posicion_tb = 4'b0010;
      #100;

      posicion_tb = 4'b0011;
      #100;

      posicion_tb = 4'b0100;
      #100;

      posicion_tb = 4'b0101;
      #100;

      posicion_tb = 4'b0110;
      #100;

      posicion_tb = 4'b0111;
      #100;

      posicion_tb = 4'b1000;
      #100;

      posicion_tb = 4'b1001;
      #100;

      posicion_tb = 4'b1010;
      #100;

      posicion_tb = 4'b1011;
      #100;
```

```
      posicion_tb = 4'b1100;
      #100;

      posicion_tb = 4'b1101;
      #100;

      posicion_tb = 4'b1110;
      #100;

      posicion_tb = 4'b1111;
      #100;

      $stop;
end

endmodule
```

RORO_Suprime.v

```
module RORO_Suprime(
    input [3:0] posicion,
    output [11:0] instruccion
);
    reg [11:0] srom [0:15];
    assign instruccion = srom[posicion];
    initial
    begin
        $readmemb("Instrucciones.txt", srom);
    end
endmodule
```

RoRO_Suprime_tb.v

```
`timescale 1ns/1ns
module RORO_Suprime_tb();

reg[3:0] posicion_tb;
wire[11:0] instruccion_tb;

RORO_Suprime DUV(
      .posicion(posicion_tb),
      .instruccion(instruccion_tb)
);

initial
begin
      posicion_tb = 4'b0000;
      #100;
```

```
      posicion_tb = 4'b0001;
      #100;

      posicion_tb = 4'b0010;
      #100;

      posicion_tb = 4'b0011;
      #100;

      posicion_tb = 4'b0100;
      #100;

      posicion_tb = 4'b0101;
      #100;

      posicion_tb = 4'b0110;
      #100;

      posicion_tb = 4'b0111;
      #100;

      posicion_tb = 4'b1000;
      #100;

      posicion_tb = 4'b1001;
      #100;

      posicion_tb = 4'b1010;
      #100;

      posicion_tb = 4'b1011;
      #100;

      posicion_tb = 4'b1100;
      #100;

      posicion_tb = 4'b1101;
      #100;

      posicion_tb = 4'b1110;
      #100;

      posicion_tb = 4'b1111;
      #100;

      $stop;
end

endmodule
```

AlU.v

```
module ALU(
    input [3:0] ALU_op,
    input [31:0] A,
    input [31:0] B,
    output reg [31:0] result,
```

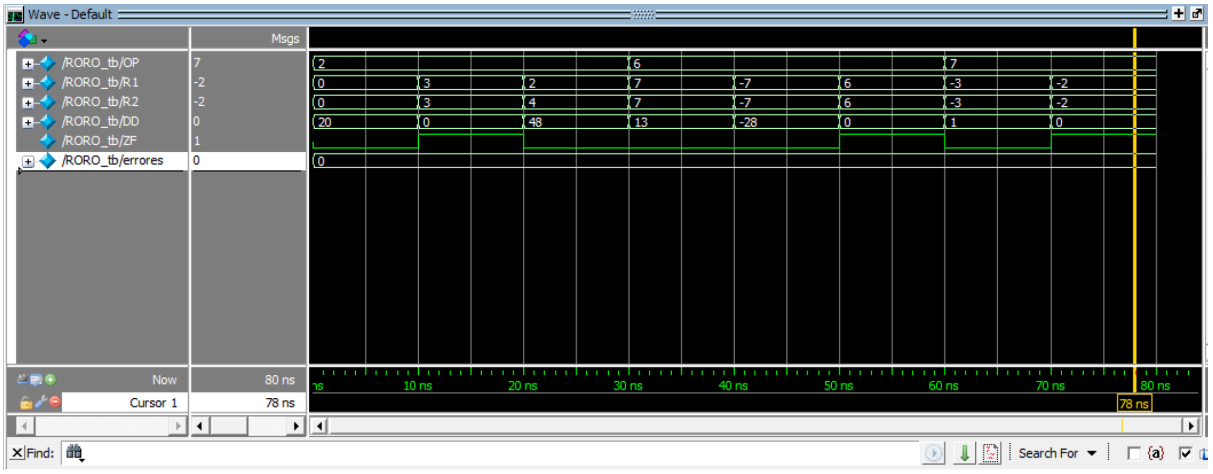
```
    output reg zero
);
    always @*
    begin
        case(ALU_op)
            4'd0:  result = A & B;
            4'd1:  result = A | B;
            4'd2:  result = A + B;
            4'd6:  result = A - B;
            4'd7:  result = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0;
            4'd12: result = ~(A | B);
            default: result = 32'd0;
        endcase
        zero = (result == 32'd0);
    end
endmodule
```

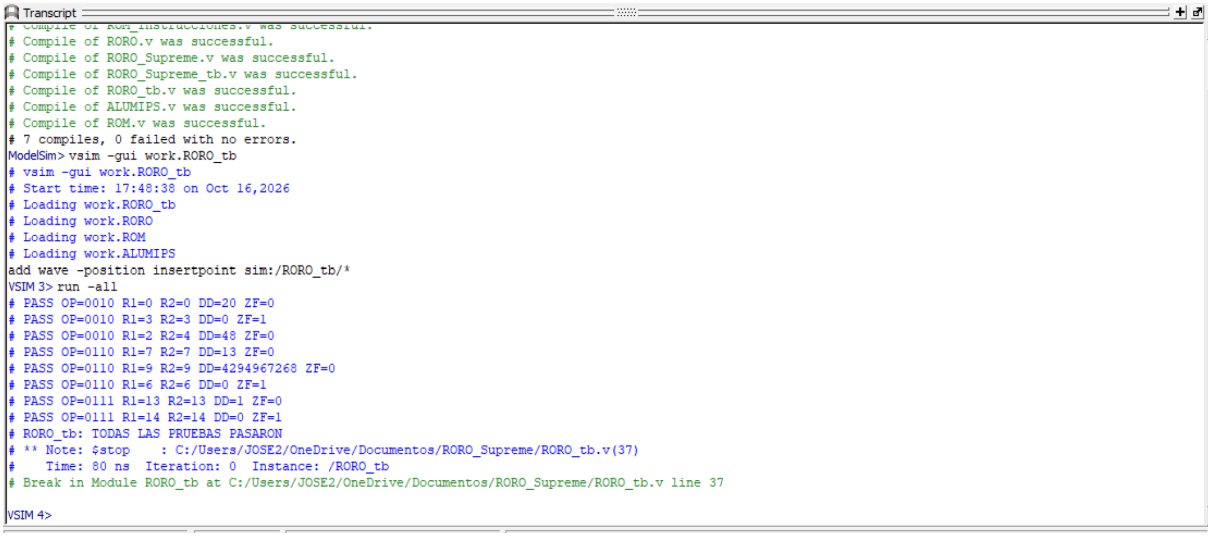
ROM.v

```
module ROma (
    input [3:0] DirL,
    output [31:0] Data
);
    reg [31:0] srom [0:15];
    assign Data = srom[DirL];
    initial
    begin
        $readmemb("Datos_ROma.txt", srom);
    end
endmodule
```

CAPTURAS

- RORO



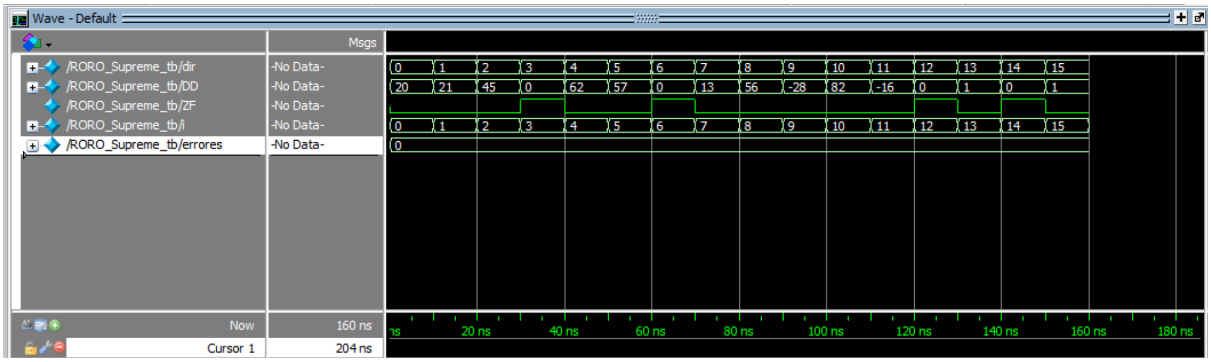


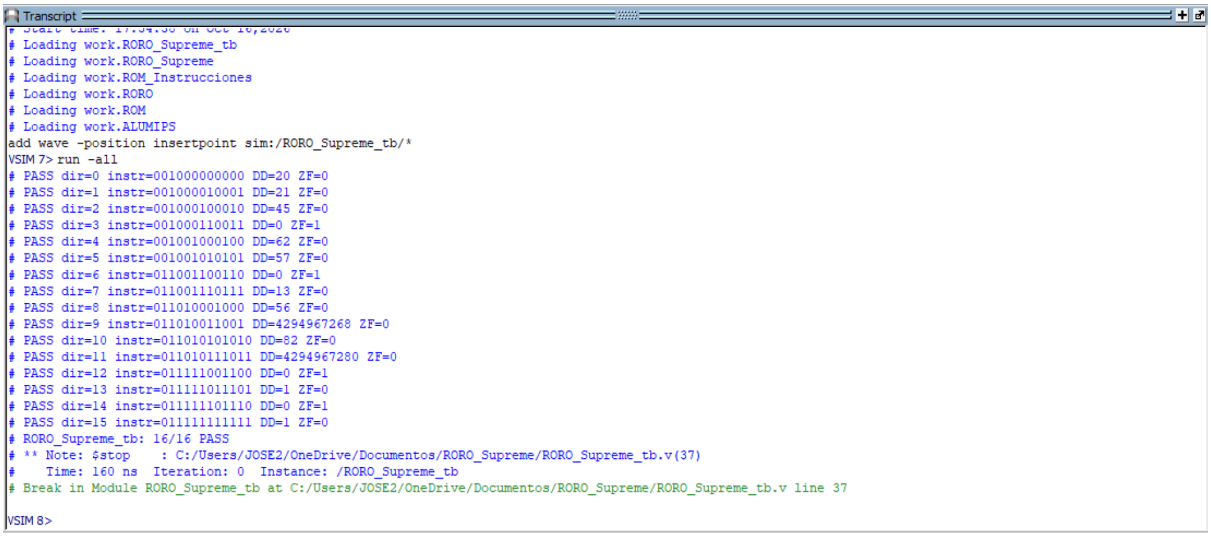
- RORO_Supreme

La simulación de RORO_Supreme_tb recorrió las 16 direcciones de instrucción. En la ventana Wave se observan DD y ZF para cada caso, el contador de errores termina en 0, por lo que las salidas coincidieron con los valores esperados definidos.

En la **dirección 9**, la resta produjo DD = −28. Como el resultado no es cero, la bandera ZF vale 0.

En la **dirección 3**, el resultado fue DD = 0; por eso, la bandera ZF vale 1.





3. Tabla de instrucciones

Dir.	OP	ROMa	ROMina	DD esperado
0	2	14	6	20
1	2	9	12	21
2	2	30	15	45
3	2	0	0	0
4	2	44	18	62
5	2	22	35	57

6	6	10	10	0
7	6	18	5	13
8	6	70	14	56
9	6	12	40	4294967268
10	6	90	8	82
11	6	5	21	4294967280
12	7	20	20	0
13	7	3	27	1
14	7	60	9	0
15	7	7	18	1

4. CONCLUSIÓNES

Yael Santero
Comprendí el funcionamiento de las memorias ROM, como interactúan y se comunican con los otros componentes además de ver varias maneras de crear una. La comunicación de diferentes componentes mediante uno con iteraciones, como se realizan y cómo funcionan, al igual de como la ALU recibe toda la información para realizar las operaciones.

Jose Nery
Logré comprender la integración sistémica entre la memoria de instrucciones, las estructuras de almacenamiento para operandos y la ALU para el procesamiento ordenado de operaciones. Al igual que la importancia de verificar de forma más funcional que cada instrucción, dirección de memoria y valor obtenido concuerden con nuestras propuestas.

Martin Geronimo
Se logró comprender la integración funcional y la verificación del procesador modular RORO Supreme. Mediante la jerarquización adecuada de componentes en Verilog, se demostró la lectura correcta de instrucciones de 12 bits, la extracción de operandos desde las memorias de datos ROMa y ROMina, y la ejecución precisa en la unidad ALUMIPS.